
Data Structures

BS (CS) _Fall_2025

Lab_08 Task



Learning Objectives:

1. Stacks and Queues using Linked Lists in C++

Lab Task 1

Scenario

Simulate a fundamental CPU scheduling algorithm used in operating systems. The Shortest Job First (SJF) algorithm selects the process with the smallest execution time to run next. Your task is to implement a SJF scheduler using a queue.

Note: This is not the accurate implementation of the SJF rather a similar approach to learn queues.

Task Requirements

Create a priority queue where each process is an object containing:

- Process ID (integer starting from 1, auto-incremented)
- Process Name (string)
- Execution Time (integer, time in seconds)
- Status (string: "Waiting", "Executing")

Core Functionalities to Implement

1. Process Management:

- `addProcess(processName, arrivalTime, executionTime)` - Adds a new process to the ready queue at the appropriate position based on the execution time.
- `executeNext()` - Removes and returns the process from the queue.
- `changeProcessTime(processID, newTime)` - Changes the execution time of a process and reorder the queue.

2. Scheduler Monitoring:

- `displayReadyQueue()` - Displays all processes in the ready queue
- `getNextProcess()` - Views the next process without removing it.

Implementation Guidelines

- Implement a `Process` class and a `SJFScheduler` class.
- The priority in this queue is determined by the `executionTime` (shorter time = higher priority).

Google Test Cases:

```
TEST(SJFSchedulerTest, AddProcessAssignsUniqueIDs) {
    SJFScheduler<int> scheduler;
    int id1 = scheduler.addProcess("P1", 0, 5);
```

```

    int id2 = scheduler.addProcess("P2", 1, 3);
    EXPECT_EQ(id1, 1);
    EXPECT_EQ(id2, 2);
}

TEST(SJFSchedulerTest, AddProcessStoresCorrectDetails) {
    SJFScheduler<int> scheduler;
    int id = scheduler.addProcess("P3", 2, 8);
    Process<int> proc = scheduler.getNextProcess();
    EXPECT_EQ(proc.processID, id);
    EXPECT_EQ(proc.processName, "P3");
    EXPECT_EQ(proc.executionTime, 8);
    EXPECT_EQ(proc.status, "Waiting");
}

TEST(SJFSchedulerTest, ShortestJobExecutesFirst) {
    SJFScheduler<int> scheduler;
    scheduler.addProcess("P1", 0, 7);
    scheduler.addProcess("P2", 1, 4);
    scheduler.addProcess("P3", 2, 2);

    Process<int> proc = scheduler.executeNext();
    EXPECT_EQ(proc.processName, "P3");
    EXPECT_EQ(proc.executionTime, 2);
    EXPECT_EQ(proc.status, "Executing");
}

TEST(SJFSchedulerTest, ExecuteNextRemovesFromQueue) {
    SJFScheduler<int> scheduler;
    scheduler.addProcess("P1", 0, 5);
    scheduler.addProcess("P2", 1, 2);

    Process<int> first = scheduler.executeNext(); // P2
    EXPECT_EQ(first.processName, "P2");

    Process<int> next = scheduler.getNextProcess(); // P1 remains
    EXPECT_EQ(next.processName, "P1");
}

TEST(SJFSchedulerTest, ExecuteNextRemovesFromQueue) {
    SJFScheduler<int> scheduler;
    scheduler.addProcess("P1", 0, 5);
    scheduler.addProcess("P2", 1, 2);

    Process<int> first = scheduler.getNextProcess(); // P1 remains

```

```

    EXPECT_EQ(first.processName, "P1");

    Process<int> second = scheduler.executeNext(); // P1
    EXPECT_EQ(second.processName, "P1");
}

TEST(SJFSchedulerTest, ChangeExecutionTimeReordersQueue) {
    SJFScheduler<int> scheduler;
    int id1 = scheduler.addProcess("P1", 0, 6);
    int id2 = scheduler.addProcess("P2", 1, 4);

    EXPECT_TRUE(scheduler.changeProcessTime(id1, 2));

    Process<int> next = scheduler.getNextProcess();
    EXPECT_EQ(next.processID, id1);
    EXPECT_EQ(next.executionTime, 2);
}

TEST(SJFSchedulerTest, ChangeExecutionTimeOfNonExistingProcessReturnsFalse) {
    SJFScheduler<int> scheduler;
    scheduler.addProcess("P1", 0, 5);
    EXPECT_FALSE(scheduler.changeProcessTime(42, 3));
}

TEST(SJFSchedulerTest, AddProcessWithZeroExecutionTimeAllowed) {
    SJFScheduler<int> scheduler;
    int id = scheduler.addProcess("P0", 0, 0);
    Process<int> proc = scheduler.getNextProcess();
    EXPECT_EQ(proc.executionTime, 0);
    EXPECT_EQ(proc.processID, id);
}

TEST(SJFSchedulerTest, MultipleEqualExecutionTimesOrderedByArrival) {
    SJFScheduler<int> scheduler;
    int id1 = scheduler.addProcess("P1", 0, 3);
    int id2 = scheduler.addProcess("P2", 1, 3);

    Process<int> first = scheduler.executeNext();
    Process<int> second = scheduler.executeNext();

    EXPECT_EQ(first.processID, id1);
    EXPECT_EQ(second.processID, id2);
}

```

Lab Task 2

Scenario

Programming languages use a call stack to manage function calls and returns. When a function is called, a stack frame is pushed. When a function returns, its frame is popped. Your task is to simulate this behavior.

Task Requirements

Create a stack where each stack frame is an object containing:

- Function Name (string)
- Line Number of Call (integer, to simulate execution order)
- Number of Parameters(integer)

Core Functionalities to Implement

1. Execution Simulation:

- `callFunction()` - Pushes a new stack frame for the function onto the call stack.
- `returnFromFunction()` - Pops the top stack frame from the call stack, simulating a function return.

2. Stack Inspection:

- `displayCallStack()` - Displays the current call stack from top (most recent) to bottom (main), showing the function name and call order.
- `getCurrentFunction()` - Returns the name of the function at the top of the stack (the currently executing function).

Implementation Guidelines

- Implement a Function and StackFrame class
- Your StackFrame class should also have a maximum limit to return false while insertion to simulate a stack overflow error.

Google Test Cases:

```
TEST(CallStackTest, CallSingleFunction) {
    CallStack<int> stack(5); // max size 5
    EXPECT_TRUE(stack.callFunction("main", 1, 0));
    EXPECT_EQ(stack.getCurrentFunction(), "main");
}

TEST(CallStackTest, CallMultipleFunctions) {
    CallStack<int> stack(5);
```

```

    stack.callFunction("main", 1, 0);
    stack.callFunction("funcA", 5, 2);
    stack.callFunction("funcB", 10, 1);

    EXPECT_EQ(stack.getCurrentFunction(), "funcB");
}

TEST(CallStackTest, ReturnFromFunction) {
    CallStack<int> stack(5);
    stack.callFunction("main", 1, 0);
    stack.callFunction("funcA", 2, 1);

    EXPECT_EQ(stack.getCurrentFunction(), "funcA");
    EXPECT_TRUE(stack.returnFromFunction());
    EXPECT_EQ(stack.getCurrentFunction(), "main");
}

TEST(CallStackTest, ReturnFromEmptyStack) {
    CallStack<int> stack(5);
    EXPECT_FALSE(stack.returnFromFunction());
}

TEST(CallStackTest, StackOverflow) {
    CallStack<int> stack(2); // max size 2
    EXPECT_TRUE(stack.callFunction("main", 1, 0));
    EXPECT_TRUE(stack.callFunction("funcA", 2, 1));
    EXPECT_FALSE(stack.callFunction("funcB", 3, 2)); // should fail
}

TEST(CallStackTest, GetCurrentFunctionEmpty) {
    CallStack<int> stack(5);
    EXPECT_EQ(stack.getCurrentFunction(), ""); // No function should be running
}

TEST(CallStackTest, NestedFunctionCallsAndReturns) {
    CallStack<int> stack(5);
    stack.callFunction("main", 1, 0);
    stack.callFunction("funcA", 2, 2);
    stack.callFunction("funcB", 5, 1);

    EXPECT_EQ(stack.getCurrentFunction(), "funcB");
    stack.returnFromFunction(); // returning from funcB
    EXPECT_EQ(stack.getCurrentFunction(), "funcA");

    stack.callFunction("funcC", 7, 3);

```

```
EXPECT_EQ(stack.getCurrentFunction(), "funcC");

stack.returnFromFunction(); // returning from funcC
EXPECT_EQ(stack.getCurrentFunction(), "funcA");

stack.returnFromFunction(); // returning from funcA
EXPECT_EQ(stack.getCurrentFunction(), "main");
}
```